COMPUTER ENGINEERING
QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 Data Structures And Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 10/16/26 | **DATE SUBMITTED:** | 10/16/26 |
| **NAME:** | Acala, Matt John C.<br>Esteta, Jesse Matthew | **INSTRUCTOR:** | Engr. Jimlord M.Quejado |

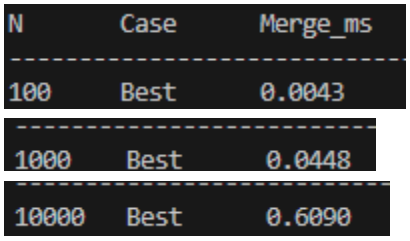
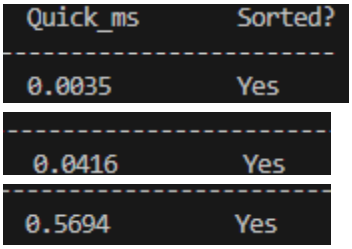
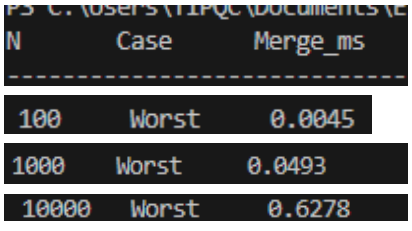
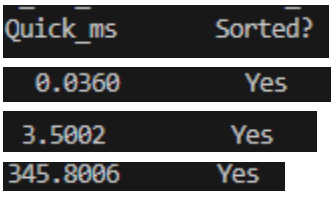
## HOA 14.1 Algorithmic Complexity

### 1. PROCEDURE OUTPUT

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

| | Best Case | Worst Case | Analysis |
|---|---|---|---|
| **Algorithm 1(Merge Sort)** | O(n) = n log n | O(n) = n log n | Merge sort has more predictable running time. |
| **Algorithm 2(Quick Sort)** | O(n) = n log n | O(n) = n^2 | Quicksort can be fast when its partitions are balanced, but this pivot choice performs poorly on sorted input. |

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

| | Algorithm 1(Merge Sort) | Algorithm 2(Quick Sort) |
|---|---|---|
| **Best Case** |  |  |
| **Worst Case** |  |  |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

# Linear search pseudocode

```
LINEAR SEARCH(array, n, key)
   FOR i = 0 TO n - 1
      IF array[i] = key
         RETURN i
   RETURN -1
```

Linear search checks one number at a time. It stops when it finds the key. If the key is missing, it checks all n numbers and returns -1. It works on sorted or unsorted arrays.

# Binary search pseudocode

```
BINARY SEARCH(sorted array, n, key)
   low = 0
   high = n - 1
   WHILE low <= high
      mid = low + FLOOR((high - low) / 2)
      IF array[mid] = key
         RETURN mid
      IF array[mid] < key
         low = mid + 1
      ELSE
         high = mid - 1
   RETURN -1
```

# Time and space complexity

| Algorithm | Best Time | Worst Time | Space Complexity |
|---|---|---|---|
| **Linear Search** | O(1) | O(n) | O(1) |
| **Binary Search** | O(1) | $O(\log_2 n)$ | O(1) and $O(\log n)$ |

| Input Size (N) | Elapsed Time for Binary Search (Worst-Case) | Elapsed Time for Linear Search (Worst-Case) |
|---|---|---|
| 1,000 | N Binary_Avg (us)<br>1000 0.0430 | Linear_Avg (us) Found?<br>2.1226 Yes |
| 10,000 | 10000 0.0595 | 21.3511 Yes |

| | | |
|---|---|---|
| **100,000** | 100000 0.0713 | 210.3638 Yes |

**Analysis:**
At 100,000 numbers, binary search took 0.0713 µs and linear search took 210.3638 µs. Linear search checked 100,000 numbers, while binary search completed 17 iterations. The gap shows why binary search is useful for large sorted arrays.

**Conclusion:**

**Acala:** This activity showed how input size and data order affect algorithm performance. Merge sort keeps O(n log n) time, while quicksort can reach O(n²) with poor partitions. Binary search needs sorted data and takes O(log n) time, while linear search takes O(n) in the worst case. Timing the algorithms helped connect theory to measured results, although short runs were affected by system overhead. The program produced correct results, but repeating tests and improving timing accuracy would strengthen the analysis.

**Esteta:** This activity showed that Merge sort has a very consistent time lapsed while quick sort's time grows bigger as input grows bigger. This showed how merge sort's O(n log n) and quick sort's O(n^2) time complexity affects algorithm performance. It also showed how long binary and linear search works on big amounts of data where binary search took a consistent time while linear search's time grew bigger as the input got larger. Timing the algorithms helped connect theoretical measure to actual result. By doing multiple runs one can compute an accurate average run time for a program/ system.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.